

Reward Information Bandwidth for Code LLM Training: A Negative Result

Anonymous Authors¹

Abstract

This paper investigates whether reward information bandwidth affects reinforcement learning convergence for code generation. Three reward functions are compared: binary pass/fail (approximately 1 bit), categorical error-type scoring (approximately 1.5 bits), and high-bandwidth combining categorical scoring with continuous test pass ratio and partial credit (approximately 2 bits). At proof-of-concept scale (1 epoch, 3 seeds per condition on CodeLlama-7B-Instruct with MBPP), all conditions produce 0.0% pass@1, preventing meaningful statistical comparison. This null result establishes that the specific experimental configuration—REINFORCE with advantage baseline, 1 epoch, approximately 94 gradient steps—is insufficient for any learning signal to emerge. The hypothesis remains untested rather than falsified. This report documents the methodology and experimental setup for future adequate-scale replication.

1. Introduction

Reinforcement learning from execution feedback (RLEF) has emerged as a paradigm for improving code generation models by providing reward signals derived from program execution outcomes. Prior work has explored various feedback granularities: binary pass/fail rewards (Le et al., 2022), continuous test pass rates (Liu et al., 2023), and multi-signal combinations incorporating error type information (Shojaee et al., 2023). Information theory suggests that higher bandwidth signals—those providing more information per gradient update—should enable more precise credit assign-

ment and faster convergence.

This work aimed to provide a controlled comparison of feedback granularities using an "information bandwidth" framework. The hypothesis was that under fixed compute budget, higher bandwidth rewards would accelerate convergence to a pass@1 threshold. However, the proof-of-concept scale experiment was insufficient to produce any learning signal across all conditions.

The result was that at proof-of-concept scale (1 epoch, 3 seeds per condition), all nine training runs produced 0.0% pass@1 on the evaluation set. No condition exhibited learning. Statistical comparison was not possible because all values were at floor with zero variance.

This null result does not falsify the information bandwidth hypothesis. It is a finding about the specific experimental setup: the configuration (REINFORCE algorithm, 1 epoch, CodeLlama-7B-Instruct, MBPP) does not produce measurable learning within the allocated compute budget.

This report documents: (1) the experimental configuration that failed to produce learning, (2) the methodology for future adequate-scale replication, and (3) a diagnostic gap—the absence of training curves prevents distinguishing between insufficient scale, catastrophic forgetting, and implementation error.

2. Related Work

2.0.1. 2.1 EXECUTION FEEDBACK FOR CODE LLMS

CodeRL (Le et al., 2022) established the RLEF paradigm using binary execution rewards with an actor-critic architecture. PPOCoder (Shojaee et al., 2023) adapted Proximal Policy Optimization for code generation with execution-based rewards. Both approaches trained for multiple epochs and report final accuracy without learning curves or samples-to-threshold metrics.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

2.0.2. 2.2 MULTI-GRANULARITY FEEDBACK

RLTF (Liu et al., 2023) introduced fine-grained feedback incorporating test pass rates and error type information, comparing it to binary rewards. Their results suggested that richer feedback improves performance. The present work builds on the multi-granularity concept but uses an "information bandwidth" framing and different infrastructure.

2.0.3. 2.3 PROCESS SUPERVISION

Process reward models (Lightman et al., 2023) demonstrated that step-by-step supervision outperforms outcome-only supervision for mathematical reasoning. This principle may transfer to code generation via execution feedback, though this requires explicit annotation in the reasoning domain versus free execution signals in code.

3. Methodology

3.0.1. 3.1 INFORMATION BANDWIDTH FRAMEWORK

Reward granularity is framed in information-theoretic terms with approximate bit estimates:

- **LOW (Binary):** $r \in 0, 1$ provides approximately 1 bit per update.
- **MEDIUM (Categorical):** Error-type scoring provides ordered feedback with approximately 1.5 bits.
- **HIGH (Bandwidth):** Composite of categorical, pass ratio, and partial credit provides approximately 2 bits.

These bit estimates are heuristic approximations; precise calculations would require entropy analysis of actual reward distributions.

3.0.2. 3.2 ERROR-TYPE SCORING

The categorical reward assigns scores based on error type, reflecting distance to correct solution:

Error Type	Score	Rationale
Syntax error	0.00	Code does not parse
Runtime error	0.25	Code runs but crashes
Assertion failure	0.50	Code runs but produces wrong output
Pass	1.00	Code is correct

3.0.3. 3.3 HIGH-BANDWIDTH REWARD COMPOSITION

The high-bandwidth condition combines three signals with fixed weights:

$$r = 0.5 \times \text{categorical_score} + 0.3 \times \text{pass_ratio} + 0.2 \times \text{partial_credit}$$

where `pass_ratio` is the fraction of test cases passed and `partial_credit` measures numeric closeness for assertion failures when expected and actual values are extractable.

3.0.4. 3.4 TRAINING CONFIGURATION

- **Model:** CodeLlama-7B-Instruct (codellama/CodeLlama-7b-Instruct-hf)
- **Training data:** MBPP sanitized split (approximately 374 problems)
- **Evaluation:** MBPP validation subset (50 problems)
- **Algorithm:** REINFORCE with advantage baseline (not PPO despite initial design)
- **Epochs:** 1 (proof-of-concept scale)
- **Seeds:** 3 per condition (9 total runs)
- **Batch size:** 4
- **Gradient steps:** Approximately 94 per epoch
- **Learning rate:** $1e-5$
- **Test timeout:** 5.0 seconds

4. Experimental Setup

4.0.1. 4.1 RESEARCH QUESTIONS

- RQ1: Does the high-bandwidth condition reach $\text{pass}@1 > 0.3$ in fewer training samples than binary?
- RQ2: Does the categorical condition outperform binary?
- RQ3: Does error-type distribution differ across conditions?

4.0.2. 4.2 CONDITIONS

Three conditions were evaluated:

1. **Binary:** reward = 1.0 if all tests pass, else 0.0
2. **Categorical:** reward = score based on worst error type (passed=1.0, assertion=0.5, runtime=0.25, syntax=0.0)
3. **High-bandwidth:** reward = $0.5 \times \text{categorical} + 0.3 \times \text{pass_ratio} + 0.2 \times \text{partial_credit}$

4.0.3. 4.3 EVALUATION METRICS

- **Primary:** pass@1 on MBPP validation subset (50 problems)
- **Statistical:** Independent t-test, Cohen’s d, significance at $p < 0.05$
- **Threshold:** pass@1 > 0.3 for convergence analysis

4.0.4. 4.4 MISSING DIAGNOSTICS

The following diagnostics were not recorded:

- Training loss curves
- KL divergence or entropy monitoring
- Intermediate checkpoints
- Zero-shot baseline verification in the evaluation setup

5. Results

5.0.1. 5.1 MAIN RESULTS

All conditions failed to exceed floor performance:

Condition	pass@1 (mean $\pm$ std)	Samples to Threshold	Seeds Reaching Threshold
Binary	0.0000 $\pm$ 0.0000	Not reached	0/3
Categorical	0.0000 $\pm$ 0.0000	Not reached	0/3
High-bandwidth	0.0000 $\pm$ 0.0000	Not reached	0/3

No condition reached the 0.3 threshold required for samples-to-threshold comparison.

5.0.2. 5.2 STATISTICAL ANALYSIS

With all values at zero and no variance, statistical comparison was mathematically undefined:

- t-statistic: NaN (undefined due to zero variance)
- p-value: NaN
- Cohen’s d: NaN
- Significant ($p < 0.05$): False

The gate evaluation failed: pass@1 for high_bandwidth (0.0000) was not greater than binary (0.0000).

5.0.3. 5.3 REWARD SIGNAL PATTERNS

Examination of raw reward trajectories from results.json shows:

- **Binary:** All 0.0 rewards across all 120 samples per seed
- **Categorical:** Some non-zero rewards (0.25) appeared after approximately 20-40 steps, increasing frequency toward end of training
- **High-bandwidth:** Some non-zero rewards (0.125) appeared with similar pattern to categorical

This suggests the model was generating some partially-correct code (runtime errors rather than syntax errors), but this did not translate to pass@1 improvement on the validation set.

5.0.4. 5.4 INTERPRETATION

The experiment cannot distinguish between:

1. Insufficient training scale (primary hypothesis)
2. Catastrophic forgetting (plausible given CodeLlama-7B-Instruct achieves approximately 30-40% pass@1 zero-shot on MBPP)
3. Implementation error (not ruled out)
4. Algorithm unsuitability (REINFORCE vs PPO)

6. Discussion

6.0.1. 6.1 KEY FINDINGS

1. **This configuration failed.** Approximately 94 gradient steps with REINFORCE on CodeLlama-7B-Instruct/MBPP produced no measurable learning as evaluated by pass@1.
1. **Hypothesis remains untested.** A null result at insufficient scale is not falsification. The theoretical mechanism (higher bandwidth enables more precise gradient directions) remains intact.
1. **Critical diagnostic gap.** Without training curves, loss plots, or intermediate checkpoints, the failure mode cannot be determined.

6.0.2. 6.2 THEORETICAL INTERPRETATION

The 0.0% pass@1 across all conditions indicates proof-of-concept scale training provides insufficient signal for policy gradient methods to learn code generation in this configuration. This is consistent with prior work—CodeRL and PPOCoder trained for multiple epochs and used different algorithmic approaches.

The theoretical mechanism remains valid:

- Binary rewards provide approximately 1 bit per feedback
- Categorical error-type scoring provides ordered feedback (syntax < runtime < assertion < pass)
- Higher bandwidth should enable more informative gradient directions

The failure is experimental (insufficient scale and/or unsuitable algorithm), not theoretical.

6.0.3. 6.3 LIMITATIONS

Decisive limitations:

- 1 epoch: approximately 94 gradient updates, insufficient for PPO-style policy improvement
- REINFORCE algorithm: higher variance than PPO, may require more samples
- No training diagnostics: cannot verify whether policy updates occurred
- No zero-shot baseline: cannot rule out catastrophic forgetting

Acknowledged limitations:

- 3 seeds: insufficient for robust variance estimation (design specified 5)
- Heuristic weighting (0.5/0.3/0.2): not validated
- Bit estimates: approximate, not rigorously calculated
- Single model size (7B)
- No hyperparameter search

6.0.4. 6.4 REQUIREMENTS FOR VALID TEST

A conclusive test of the information bandwidth hypothesis would require:

- At least 3 epochs or training until validation plateau
- PPO or lower-variance algorithm
- Training curves showing policy improvement (or lack thereof)
- Zero-shot baseline verified in same evaluation setup
- At least 5 seeds per condition
- Intermediate checkpoints for failure mode diagnosis

7. Conclusion

This work attempted to test whether information bandwidth in reward functions affects code LLM training convergence. At proof-of-concept scale with REINFORCE, all conditions produced 0.0% pass@1, preventing any comparison.

This is a documentation of experimental failure, not a research contribution. The experiment cannot claim to have established minimum scale requirements because the diagnostics necessary to determine why the experiment failed were not recorded.

The information bandwidth hypothesis remains open. Future work should use adequate scale, include training diagnostics, and verify the experimental setup produces learning before attempting reward ablation.

References

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. *arXiv preprint arXiv:2306.09896*, 2023.
- Face, H. Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2023.
- Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. H. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35, 2022.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Liu, J., Xia, Y., Wang, Y., Zhang, Y., Wang, C., Ding, Y., and Xu, J. Rlhf: Reinforcement learning from unit test feedback. *arXiv preprint arXiv:2307.04349*, 2023.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin,

J., et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Shojaee, P., Jain, A., Fernandes, P., Perez, E., Mao, J., Xiong, H., and Neubig, G. Execution-based code generation using deep reinforcement learning. *arXiv preprint arXiv:2306.05826*, 2023.